HTTP Keep-Alive vs New-Connection Requests

Recorded technical presentation · narration script · ITUC201 · Computer Networks · NetVista 3D · AY 2026–27

Video length	13:30 (advanced investigation band: 10–15 min)
Live site	https://reuse-reconnect.netlify.app
Code	https://github.com/kismats1107-stack/reuse-reconnect
Presenter	[your name] · student ID [your ID]

How to record (hands-free: you only read)

1. Open **OBS Studio** once and leave it open. The autopilot creates its scenes and starts, pauses and stops the recording by itself.
2. In a terminal in the project folder, run `npm run video`. The website opens full-screen and a **Teleprompter** window appears at the top of the screen, under your webcam.
3. Click **Start recording** in the teleprompter. From then on you only read. The text **follows your voice**: words you have said turn bright, the next word is highlighted, and it scrolls only when you reach the next line. A segment moves on (after a 3-2-1 countdown) only once you have said its last lines and the demo has finished. The label at the top says *follows your voice*; if it says *timer*, voice-follow is unavailable (it needs internet) and a slow timer is used instead.
4. Need more time? Press **Hold** (or Space) and press it again when ready. Finished early? Press **Next** (or the right arrow key).
5. After segment 8 the recording stops and the teleprompter shows where the video is saved. Upload it to YouTube as **Unlisted**.

> **The teleprompter, this PDF and the terminal are never recorded.** OBS records only the website window (and Wireshark in segment 6), plus your face-cam in the bottom-right corner. Speak calmly, about 2 words per second. Fill in the [highlighted] words first.

Rubric coverage (20 marks)

Criterion	Marks	Where the video earns it
Technical accuracy and conceptual understanding	5	2 Theory · 8 Conclusion & exam answer
Working visualisation / simulation or executable demonstration	5	3 Guided tour · 4 Experiments
Technical explanation, experimentation and interpretation	4	4 Experiments · 5 Graphs · 6 Real measurement
Recorded individual technical presentation	3	Your own voice throughout · 1 Introduction
Supporting files and reproducibility	3	6 Wireshark capture · 7 GitHub, tests, README

Timeline

#	Segment	Time	Length	Rubric
1	Introduction	0:00–0:40	0:40	Presentation
2	Essential theory	0:40–2:40	2:00	Technical accuracy
3	Working simulation: the guided tour	2:40–6:10	3:30	Working visualisation
4	Experiments: changing the conditions	6:10–7:40	1:30	Experimentation
5	Results and graphs	7:40–9:10	1:30	Interpretation
6	Real measurement: Python + Wireshark	9:10–11:40	2:30	Experimentation · Evidence
7	Supporting files and reproducibility	11:40–12:40	1:00	Supporting files
8	Conclusion and exam answer	12:40–13:30	0:50	Technical accuracy

0:00–0:40 1 · Introduction PRESENTATION

On screen (autopilot):

- Results page, top: the 3D towers slowly turning

Hello everyone. I am [your name], student ID [your ID].

My Computer Networks investigation is HTTP Keep-Alive versus new-connection requests. The question is: when a web page needs many files, how does reusing one TCP connection, instead of opening a new one for every file, change the number of connections, the number of handshakes, and the time the page takes to load?

I built an interactive 3D simulation, measured the same thing with a real Python server and Wireshark, and I will show you both.

85 words · about 39 s of speech in a 0:40 segment

0:40–2:40 2 · Essential theory

TECHNICAL ACCURACY

On screen (autopilot):

- Glossary cards flip: Handshake, SYN, RTT, Keep-Alive
- Sequence diagrams: 3 files, 50 ms, packets play
- Formula card

First, the theory. A web page is many files: HTML, CSS, scripts and images. Each one is a separate HTTP request, and HTTP runs over TCP.

Before any data moves, TCP opens a connection with a three-way handshake. The client sends SYN, the server replies SYN-ACK, and the client answers ACK. The SYN has to reach the server and the SYN-ACK has to come back, so every handshake costs one full round-trip time, the RTT, before the first file can even be requested.

With the header Connection: close, the server closes the connection after every response, so the next file needs a brand-new connection and a new handshake. In HTTP/1.1, connections are persistent by default. We call this Keep-Alive: one handshake, and then every request travels over the same connection.

These sequence diagrams come from my model, with three files at fifty milliseconds. Time runs downwards. On the left, every file starts with SYN, SYN-ACK, ACK. On the right, the handshake happens once, and then it is only GET and 200 OK. Both use the same scale, so the right-hand diagram finishing earlier is exactly the time saved.

As formulas: with N files, a new connection per file takes about N times two RTT, Keep-Alive takes one RTT plus N times RTT, and the saving is N minus one, times the RTT. The transfer time is the same in both, so it cancels out.

234 words · about 106 s of speech in a 2:00 segment

2:40–6:10 3 · Working simulation: the guided tour WORKING VISUALISATION

On screen (autopilot):

- 3D Lab, guided tour (voice muted: you are the narrator)
- Lesson 1: new connection per file, one packet at a time
- Lesson 2: Keep-Alive
- Lesson 3: race with 10 files at 50 ms

Now the working simulation. This is my 3D lab, built with React and Three.js. The laptop on the left is the browser, the tower on the right is the web server, and every shape flying between them is a packet generated by my model. Orange diamonds are the handshake, blue cubes are requests, green cubes are the files coming back, and grey rings close the connection.

The guided tour sends one packet at a time. Lesson one is the old way. The browser sends SYN: can we connect? The server answers SYN-ACK. The browser sends ACK together with its first GET. The server sends index.html back and then closes the connection with FIN.

For the second file the whole handshake happens again, and the used connection is thrown onto this pile next to the browser. Two files: two connections and two handshakes.

Lesson two is Keep-Alive, with the same two files. After the first file, look at the message: same connection, next file please. There is no new handshake, and the connection label says reused. Two files: one connection and one handshake.

Lesson three races both with ten files at fifty milliseconds. Watch the handshake counters at the top: lane A climbs to ten, lane B stays at one. The new-connection page loads in 1042 milliseconds and Keep-Alive in 592. That is 450 milliseconds saved, 43 percent faster.

The tour can also narrate itself with a built-in voice and run hands-free, so students can learn from it on their own. For this video, I am the narrator.

257 words · about 117 s of speech in a 3:30 segment

6:10–7:40 4 · Experiments: changing the conditions EXPERIMENTATION

On screen (autopilot):

- Free play: 40 files, other country (100 ms), race
- 1 file, race: a tie
- Panels hidden, world rotated

Now my own experiments. I change the page to forty files and put the server in another country, one hundred milliseconds away, and race again.

Forty handshakes against one. The page takes 8174 milliseconds with new connections and 4274 with Keep-Alive. That saves 3900 milliseconds, 48 percent, which is exactly thirty-nine times one hundred milliseconds.

Now with one file only. Both finish at the same time, because with one file there is nothing to reuse. So the benefit grows with the number of files and with the distance to the server.

The panels can be hidden to see the whole 3D world, which can be rotated and zoomed.

108 words · about 49 s of speech in a 1:30 segment

7:40–9:10 5 · Results and graphs INTERPRETATION

On screen (autopilot):

- Results: 3D towers at 40 files, 100 ms
- Four graphs
- Formula with live numbers

The results page turns this into evidence. Each 3D tower is one page load, built from blocks in time order: orange for waiting on a handshake, blue for fetching a file, so height is time. With forty files at one hundred milliseconds, tower A is full of orange, tower B has a single orange block, and the outlined block on top of B is the 3900 milliseconds saved.

The graphs show the same model from four sides. Load time grows with every file, but faster for new connections. The saving grows with distance. Handshakes grow one per file for A and stay at one for B. And in the breakdown, only the striped handshake part differs.

The formula card plugs in my numbers: thirty-nine times one hundred is 3900 milliseconds.

130 words · about 59 s of speech in a 1:30 segment

9:10–11:40 6 · Real measurement: Python + Wireshark

EXPERIMENTATION · EVIDENCE

On screen (autopilot):

- Results: Real proof section, 50 ms
- Wireshark: fresh capture with the SYN filter applied

A simulation alone is not proof, so I measured it on real sockets. My Python lab runs a real HTTP/1.1 server and a raw-socket client, fetches the same files both ways, and captures the packets with Wireshark's dumpcap.

Across 200 trials, every connection was counted three independent ways: by the client, by the server, and from the SYN packets in the capture. All three agree in every trial.

On this chart the dots are real medians and the dashed lines are the model. For twenty files at fifty milliseconds, the real page took 2170 milliseconds with new connections and 1134 with Keep-Alive. That is 1035 milliseconds saved, against 950 predicted. The extra is real per-connection work that the model leaves out, like creating sockets and starting a server thread.

One honest note: a laptop cannot be fifty milliseconds away from itself, so those delays are emulated by a small relay and labelled as emulated. At zero milliseconds, on plain loopback, Keep-Alive still won: 93 against 19 milliseconds for forty files.

Here is a capture I just made with one command, five files each way. In Wireshark, the filter tcp.flags.syn equals one and tcp.flags.ack equals zero shows only connection openings: seven packets. Five for the old way, one for Keep-Alive, and one warm-up request. One SYN per connection, exactly as the model says.

222 words · about 101 s of speech in a 2:30 segment

11:40–12:40 7 · Supporting files and reproducibility SUPPORTING FILES

On screen (autopilot):

- GitHub repository: files, README, green test check

Everything here is reproducible. The full project is on GitHub: the React source code, the Python lab, 82 automated tests, the measurement data, and the Wireshark capture in the evidence folder.

The README explains how to run it with npm install and npm run dev, and how to repeat the measurement with one Python command. Every push runs the tests automatically, and the green check shows they pass. The live site is at reuse-reconnect dot netlify dot app.

78 words · about 35 s of speech in a 1:00 segment

12:40–13:30 8 · Conclusion and exam answer TECHNICAL ACCURACY

On screen (autopilot):

- Results page, top: the 3D towers

To conclude. With Connection: close, N files need N TCP connections and N handshakes. With Keep-Alive, they need one connection and one handshake. The number of HTTP requests stays N in both cases.

The time saved is about N minus one times the RTT, so it grows with more files and with distance, and I confirmed it on real sockets and in Wireshark.

As an exam answer: persistent connections reduce latency by removing repeated TCP handshakes, saving about one round-trip time for every reused request, and they also reduce load on the server, because fewer connections are opened and closed. Thank you for watching.

104 words · about 47 s of speech in a 0:50 segment